

COMS BC1016

Introduction to Computational Thinking and Data Science

# Lecture 9: Conditionals and Iteration

Sep 30, 2025

Copyright © 2026 Barnard College

October 7, 2026

# Booleans and Comparisons

# Boolean Data Type

- Booleans are data types for truth values: **True** or **False**
  - **True** is equivalent to `1`
  - **False** is equivalent to `0`
- `bool(x)` turns `x` into a boolean
  - e.g., `bool(1)` evaluates to **True** and `bool(0)` evaluates to **False**

# Comparison Operators

| Operation | Meaning                  |
|-----------|--------------------------|
| >         | greater than             |
| >=        | greater than or equal to |
| <         | less than                |
| <=        | less than or equal to    |
| ==        | equal to                 |
| !=        | not equal to             |

# Comparison Operators

| Example    | Result |
|------------|--------|
| '4' == 4   |        |
| 1 == 4 - 3 |        |
| 2 != 2     |        |
| 3 < 10 > 5 |        |

# Comparison Operators

| Example    | Result |
|------------|--------|
| '4' == 4   | False  |
| 1 == 4 - 3 | True   |
| 2 != 2     | False  |
| 3 < 10 > 5 | True   |

# Comparisons with Arrays

- Single values can be compared against each element in an array
- Comparing two arrays will compare element-by-element

DEMO

# and, or, and not

- You can combine conditional statements using **and** & **or**
  - **and** will return **True** if **all** expressions are **True** (and **False** otherwise)
  - **or** will return **True** if **any** expression is **True** (and **False** otherwise)
- You can negate a boolean value using **not**
  - **not True** will evaluate to **False**
  - **not False** will evaluate to **True**

# and, or, and not

| Example               | Result       |
|-----------------------|--------------|
| <b>True and True</b>  | <b>True</b>  |
| <b>True and False</b> | <b>False</b> |
| <b>True or False</b>  | <b>True</b>  |
| <b>False or False</b> | <b>False</b> |
| <b>not False</b>      | <b>True</b>  |

# Aggregating Comparisons

- Summing an array or list of bool values will count the **True** values only

| Example                                              | Result |
|------------------------------------------------------|--------|
| <b>True</b> + <b>False</b> + <b>True</b>             | 2      |
| 1 + 0 + 1                                            | 2      |
| sum ( [ <b>True</b> , <b>False</b> , <b>True</b> ] ) | 2      |

# Practicing Aggregating Comparisons

```
z = make_array('Tim', 'John', 'Tim')
```

| Comparison Expression                       | Value |
|---------------------------------------------|-------|
| <code>z == 'Tim'</code>                     |       |
| <code>sum(z == 'Tim')</code>                |       |
| <code>np.arange(3) &gt;= 1</code>           |       |
| <code>sum(np.arange(3) &gt;= 1)</code>      |       |
| <code>sum(np.arange(4) &lt;= 1) == 2</code> |       |

# Practicing Aggregating Comparisons

```
z = make_array('Tim', 'John', 'Tim')
```

| Comparison Expression                       | Value                                   |
|---------------------------------------------|-----------------------------------------|
| <code>z == 'Tim'</code>                     | <code>array([True, False, True])</code> |
| <code>sum(z == 'Tim')</code>                |                                         |
| <code>np.arange(3) &gt;= 1</code>           |                                         |
| <code>sum(np.arange(3) &gt;= 1)</code>      |                                         |
| <code>sum(np.arange(4) &lt;= 1) == 2</code> |                                         |

# Practicing Aggregating Comparisons

```
z = make_array('Tim', 'John', 'Tim')
```

| Comparison Expression                       | Value                                   |
|---------------------------------------------|-----------------------------------------|
| <code>z == 'Tim'</code>                     | <code>array([True, False, True])</code> |
| <code>sum(z == 'Tim')</code>                | 2                                       |
| <code>np.arange(3) &gt;= 1</code>           |                                         |
| <code>sum(np.arange(3) &gt;= 1)</code>      |                                         |
| <code>sum(np.arange(4) &lt;= 1) == 2</code> |                                         |

# Practicing Aggregating Comparisons

```
z = make_array('Tim', 'John', 'Tim')
```

| Comparison Expression                       | Value                                   |
|---------------------------------------------|-----------------------------------------|
| <code>z == 'Tim'</code>                     | <code>array([True, False, True])</code> |
| <code>sum(z == 'Tim')</code>                | 2                                       |
| <code>np.arange(3) &gt;= 1</code>           | <code>array([False, True, True])</code> |
| <code>sum(np.arange(3) &gt;= 1)</code>      |                                         |
| <code>sum(np.arange(4) &lt;= 1) == 2</code> |                                         |

# Practicing Aggregating Comparisons

```
z = make_array('Tim', 'John', 'Tim')
```

| Comparison Expression                       | Value                                   |
|---------------------------------------------|-----------------------------------------|
| <code>z == 'Tim'</code>                     | <code>array([True, False, True])</code> |
| <code>sum(z == 'Tim')</code>                | 2                                       |
| <code>np.arange(3) &gt;= 1</code>           | <code>array([False, True, True])</code> |
| <code>sum(np.arange(3) &gt;= 1)</code>      | 2                                       |
| <code>sum(np.arange(4) &lt;= 1) == 2</code> |                                         |

# Practicing Aggregating Comparisons

```
z = make_array('Tim', 'John', 'Tim')
```

| Comparison Expression                       | Value                                   |
|---------------------------------------------|-----------------------------------------|
| <code>z == 'Tim'</code>                     | <code>array([True, False, True])</code> |
| <code>sum(z == 'Tim')</code>                | 2                                       |
| <code>np.arange(3) &gt;= 1</code>           | <code>array([False, True, True])</code> |
| <code>sum(np.arange(3) &gt;= 1)</code>      | 2                                       |
| <code>sum(np.arange(4) &lt;= 1) == 2</code> | True                                    |

# Control Statements

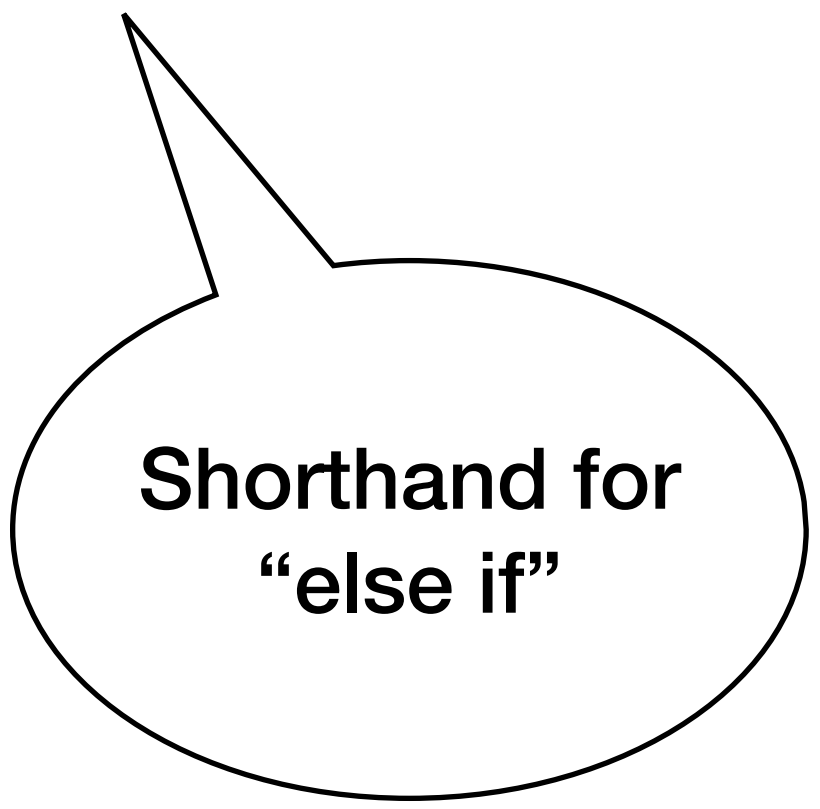
# Control Statements

**Control Statements** modify *if* and/or *how many times* a block of code is executed in a program

- Two major types are **if** and **for**
  - **if** statements specify code that should be run conditioned on something being true
    - They can also specify if alternative code should be run otherwise
  - **for** loops allow executing code over each element in some sequence of items

# **if** statements

- Conditionals begin with an **if** followed by a boolean statement
  - Runs code based on whether a boolean statement evaluates to **True**
- Conditionals can include a combination of **if**, **elif**, and **else** clauses
  - Maximum of one **if** and one **else**



Shorthand for  
“else if”

# **if** statements

```
if statement_1:
```

```
    first_code_block
```

# **if** statements

```
if statement_1:
```

```
    first_code_block
```

```
elif statement_2:
```

```
    second_code_block
```

# **if** statements

**if** statement\_1:

first\_code\_block

**else:**

second\_code\_block

# **if** statements

**if** statement\_1:

first\_code\_block

**elif** statement\_2:

second\_code\_block

**else:**

third\_code\_block

# **if** statements

**if** statement\_1:

first\_code\_block

**elif** statement\_2:

second\_code\_block

**elif** statement\_3:

third\_code\_block

**else:**

fourth\_code\_block

# Demo: If Statements

# For Loops

# `for` Statements

- `for` is a keyword that begins a multiline `for` statement
- Executing a `for` statement performs a computation for every element in a list, array, or other iterable
  - In this class, this will typically be an array
- A common special case is to perform a computation a fixed number of times

# Anatomy of a **for** Loop

Example: a **for** statement runs code with each element in an array

Variable name

Array of values

```
for item in some_array:
```

Code to evaluate in each iteration of the `for` loop

```
x = 'Here is a ' + str(item)  
  
print(x)
```

# Demo: For Loops

# Simulation

# General Process for Simulations

1. Figure out what you want to simulate
  - Example: Outcomes of a coin toss



# General Process for Simulations

1. Figure out what you want to simulate
  - Example: Outcomes of a coin toss
2. Write a function whose output is the outcome of a single simulation



# General Process for Simulations

1. Figure out what you want to simulate
  - Example: Outcomes of a coin toss
2. Write a function whose output is the outcome of a single simulation
3. Repeat the simulation for some number of iterations
  - Keep track of the results of every iteration in an array



# General Process for Simulations

1. Figure out what you want to simulate
  - Example: Outcomes of a coin toss
2. Write a function whose output is the outcome of a single simulation
3. Repeat the simulation for some number of iterations
  - Keep track of the results of every iteration in an array
4. Add results array to a table so you can plot the results



# General Process for Simulations

1. Figure out what you want to simulate
  - Example: Outcomes of a coin toss
2. Write a function whose output is the outcome of a single simulation
3. Repeat the simulation for some number of iterations
  - Keep track of the results of every iteration in an array
4. Add results array to a table so you can plot the results



Typically what we want to know about is influenced by **chance** or **randomness**

# General Process for Simulations

1. Figure out what you want to simulate
  - Example: Outcomes of a coin toss
2. Write a function whose output is the outcome of a single simulation
3. Repeat the simulation for some number of iterations
  - Keep track of the results of every iteration in an array
4. Add results array to a table so you can plot the results



To keep track of our results, we will want to **append** (add to the end) elements onto our arrays

# Random Selection

To select uniformly at random from array `some_array`

- `np.random.choice(some_array)`

To select `n` number of random elements from array `some_array`

- `np.random.choice(some_array, n)`

Note: Random does not mean arbitrary.

We mean each output has some chance of happening (**probability**)

# Adding Data to An Array

We can use `np.append` to add values onto the end of an array

- `np.append(array_1, value)`
  - Creates a new array with `value` appended to the end of `array_1`
- `np.append(array_1, array_2)`
  - Creates a new array with the values in `array_2` appended to the end of `array_1`

# Next Class

- Today
  - Conditionals
  - For Loops
- Monday
  -